THE LAWS OF SOFTWARE TEAMS

Conway's law

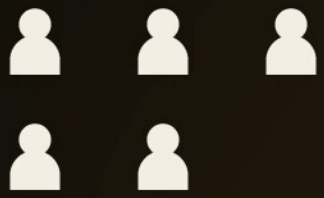
Organizations design systems that copy their communication structure

swipe →

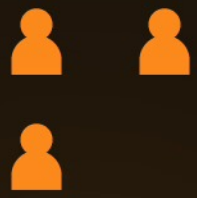
Ivan Rykov

1968 · 8 people, 2 compilers

Teams in
Same shape out



COBOL · 5 people



ALGOL · 3 people

phase 1	phase 1
phase 2	phase 2
phase 3	phase 3
phase 4	
phase 5	

Melvin Conway, “How Do Committees Invent?”, Datamation, 1968

Organizations design systems that copy their own communication structure

Conway’s law · paraphrased from the 1968 paper

architecture



Teams that don't talk ship code that doesn't fit



Glue code piles up in between

Microsoft · Windows Vista

The org chart predicted the bugs

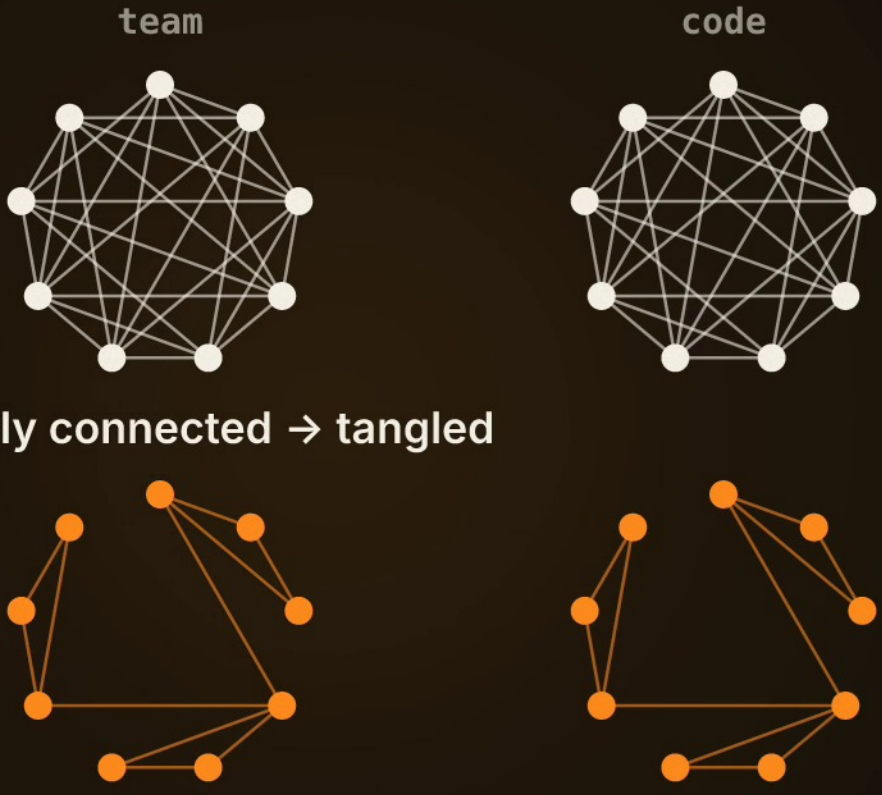


3,404 binaries · 50M+ lines of code

86% precision from org structure, better than churn, complexity, coverage

Nagappan, Murphy & Basili, ICSE 2008 · 1 dot ≈ 10 binaries

Your code is a mirror



MacCormack, Rusnak & Baldwin · HBS & MIT · diagram illustrative

Flip it: design the teams first



Amazon, ~2002: teams talk only through service interfaces

inverse Conway maneuver: LeRoy & Simons, 2010 · Amazon: Yegge, 2011

Flo: logic moved to where the team is



each team: 1 iOS · 1 Android · 2-4 backend

app store release — every few weeks

backend-driven releases

20-30 a day

via manager.dev

Want a different architecture?



Draw a different org chart

HOW TO USE IT

1. **Build each team around a product or a domain: the problem it solves and the result it owns. A team should go from an idea to a working change without waiting for other teams. In product work, cross-functional teams usually work better than separate frontend, backend and QA teams**
2. **Give every service, module and piece of shared code one owner: a team or a few named people, with the authority and the time to support it. When ownership moves, set a clear date and do a full handover. Open-ended "both teams own it" means nobody owns it. I have seen shared services that nobody owned**

HOW TO USE IT

3. Let dependent teams talk directly about contracts and shared decisions. Write decisions and their reasons where everyone can see them. Cut meetings where people only report status
4. When most tasks need several teams, first find where work gets stuck. Then review the boundaries
5. Let each team release and roll back its own changes. I have seen a shared release by checklist make every team wait for the others. When possible, change contracts between teams step by step, without breaking the other side
6. Put ownership before services. A modular monolith with clear owners is a good default for a small team. A new service adds work, so add it only when its benefit is clear

HOW TO USE IT

7. **When several teams repeat the same work, create a platform team that helps them do it on their own. Keep approvals only for changes that can cause serious damage**

With AI agents, a small team can write more code, but people still have to review it and run the system. Widen a team's ownership only when it can handle review and support. If a decision needs constant agreement between teams, writing code faster does not remove that dependency

Sources: Melvin Conway, Datamation, 1968 · LeRoy and Simons, 2010 · Nagappan, Murphy and Basili, 2008 · MacCormack, Rusnak and Baldwin · Yegge on Amazon · manager.dev on Flo

next: Dunbar’s number